Progress and Preservation

Concepts of Programming Languages

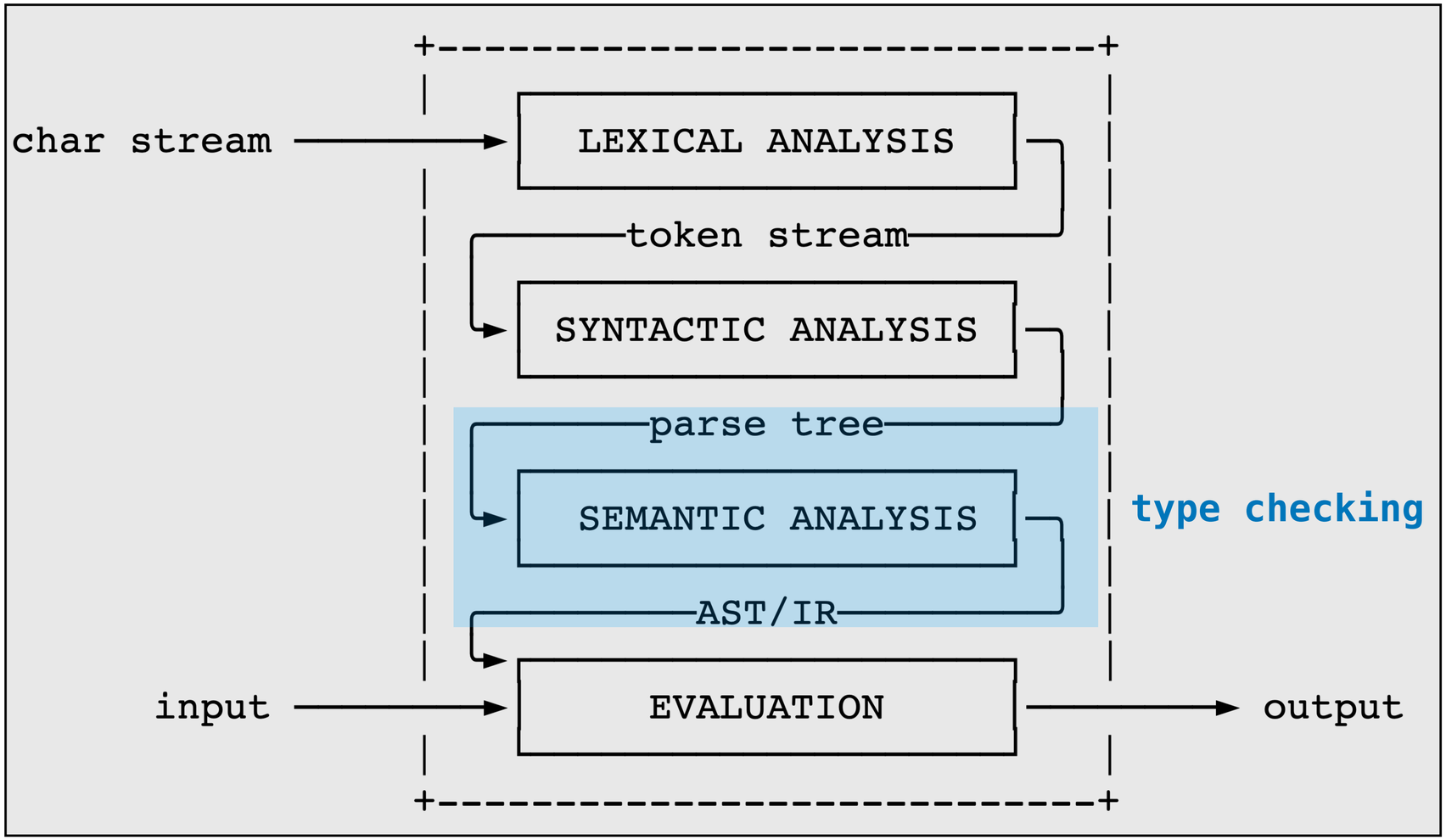
CAS CS 320

Outline

- Discuss **induction** over derivations
- Show that STLC satisfies **progress** and **preservation**

Recap

Recall: The Picture



Recall: Type Checking/Inference

```
type_check : expr -> ty -> bool
type_of : expr -> ty option
```

Recall: Type Checking/Inference

```
type_check : expr -> ty -> bool
type_of : expr -> ty option
```

Type checking: determining whether an expression can be typed

Recall: Type Checking/Inference

```
type_check : expr -> ty -> bool
type_of : expr -> ty option
```

Type checking: determining whether an expression can be typed

Type inference: *synthesizing* a type for an expression

Recall: Type Checking/Inference

```
type_check : expr -> ty -> bool

type_of : expr -> ty option
```

Type checking: determining whether an expression can be typed

Type inference: *synthesizing* a type for an expression

Theoretically, these two problems can be very different. *For STLC, they are both easy*

Recall: Syntax (STLC)

Reminder:
STLC has
type uniqueness

type annotations

unit

$$e ::= \boxed{\bullet} \mid x \mid \lambda x^{\boxed{\tau}}.\, e \mid ee$$

$$\tau ::= \boxed{\top} \mid \tau \to \tau$$

unit type

$$x ::= variables$$

The syntax is the same as that of the lambda calculus except:

» we include a unit expression

» we have types, which annotate arguments

Recall: Typing (STLC)

$$\frac{}{\Gamma \vdash \bullet : \top}\ \texttt{unit}$$

$$\frac{\Gamma, x : \tau \vdash e : \tau'}{\Gamma \vdash \lambda x^{\tau}.\, e : \tau \to \tau'}\ \texttt{abstraction}$$

$$\frac{(x : \tau) \in \Gamma}{\Gamma \vdash x : \tau}\ \texttt{variable}$$

$$\frac{\Gamma \vdash e_1 : \tau \to \tau' \qquad \Gamma \vdash e_2 : \tau}{\Gamma \vdash e_1 e_2 : \tau'}\ \texttt{application}$$

These rules enforce that a function can only be applied if we *know* that it's a function

Recall: Semantics (STLC)

$$\frac{e_1 \longrightarrow e_1'}{e_1 e_2 \longrightarrow e_1' e_2}\ \texttt{leftEval}$$

$$\frac{}{(\lambda x\,.\,e)e' \longrightarrow [e'/x]e}\ \texttt{beta}$$

Recall: Semantics (STLC)

$$\frac{e_1 \longrightarrow e_1'}{e_1 e_2 \longrightarrow e_1' e_2}\ \texttt{leftEval} \qquad\qquad \frac{}{(\lambda x\,.\,e)e' \longrightarrow [e'/x]e}\ \texttt{beta}$$

We can use any semantics (this is small-step CBN)

Recall: Semantics (STLC)

$$\frac{e_1 \longrightarrow e_1'}{e_1 e_2 \longrightarrow e_1' e_2}\ \texttt{leftEval} \qquad\qquad \frac{}{(\lambda x\,.\,e)e' \longrightarrow [e'/x]e}\ \texttt{beta}$$

We can use any semantics (this is small-step CBN)

This is part of the point. Type-checking only determines *whether* we go on to evaluate the program

Recall: Semantics (STLC)

$$\frac{e_1 \longrightarrow e_1'}{e_1 e_2 \longrightarrow e_1' e_2}\ \texttt{leftEval} \qquad\qquad \frac{}{(\lambda x\,.\,e)e' \longrightarrow [e'/x]e}\ \texttt{beta}$$

We can use any semantics (this is small-step CBN)

This is part of the point. Type-checking only determines *whether* we go on to evaluate the program

It doesn't determine *how* we evaluate the program

Practice Problem

$$(\lambda x^{(\top \to \top) \to \top} . x(\lambda z^{\top} . x(wz)))y$$

Determine the smallest context such that the above expression is typeable (also give its type)

Answer

$$(\lambda x^{(\mathsf{T}\to\mathsf{T})\to\mathsf{T}}.x(\lambda z^{\mathsf{T}}.x(wz)))y$$

$$\{y : (\mathsf{T}\to\mathsf{T})\to\mathsf{T},\ w : \mathsf{T}\to(\mathsf{T}\to\mathsf{T})\} \vdash (\lambda x^{(\mathsf{T}\to\mathsf{T})\to\mathsf{T}}.x(\lambda z^{\mathsf{T}}.x(wz)))y : \mathsf{T}$$

T

$(\mathsf{T}\to\mathsf{T})\to\mathsf{T}$

T

$(\mathsf{T}\to\mathsf{T})\to\mathsf{T}$

$(\mathsf{T}\to\mathsf{T})$

$(\mathsf{T}\to\mathsf{T})$

$\mathsf{T}\to(\mathsf{T}\to\mathsf{T})$

Induction over Derivations

The Key Idea

The Key Idea

$$\dfrac{\dfrac{}{\{\} \vdash \texttt{2} : \texttt{int}}\,(\text{intLit}) \qquad \dfrac{\dfrac{}{\{\texttt{y} : \texttt{int}\} \vdash \texttt{y} : \texttt{int}}\,(\text{var}) \qquad \dfrac{}{\{\texttt{y} : \texttt{int}\} \vdash \texttt{y} : \texttt{int}}\,(\text{var})}{\{\texttt{y} : \texttt{int}\} \vdash \texttt{y} + \texttt{y} : \texttt{int}}\,(\text{intAdd})}{\{\} \vdash \texttt{let y = 2 in y} + \texttt{y} : \texttt{int}}\,(\text{let})$$

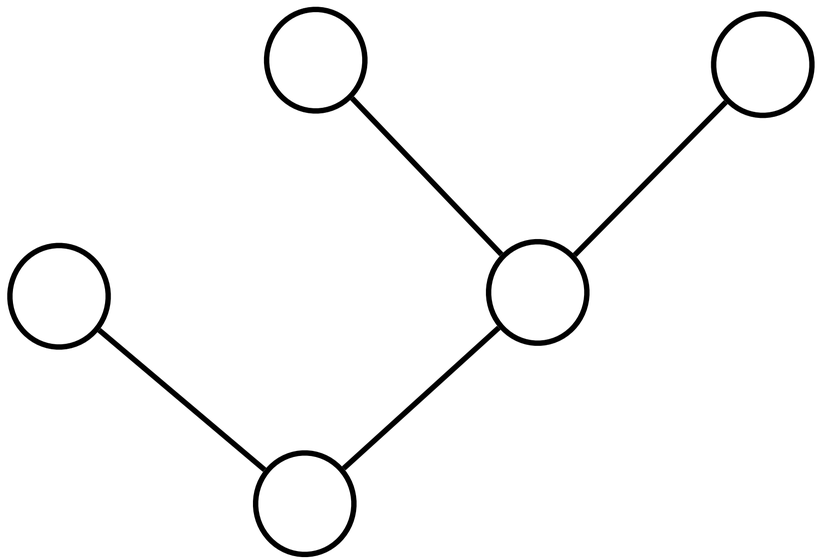
Derivations are *trees*

The Key Idea

$$\dfrac{\dfrac{}{\{\} \vdash \texttt{2} : \texttt{int}}\,(\text{intLit}) \qquad \dfrac{\dfrac{}{\{\texttt{y} : \texttt{int}\} \vdash \texttt{y} : \texttt{int}}\,(\text{var}) \qquad \dfrac{}{\{\texttt{y} : \texttt{int}\} \vdash \texttt{y} : \texttt{int}}\,(\text{var})}{\{\texttt{y} : \texttt{int}\} \vdash \texttt{y} + \texttt{y} : \texttt{int}}\,(\text{intAdd})}{\{\} \vdash \texttt{let y = 2 in y} + \texttt{y} : \texttt{int}}\,(\text{let})$$

Derivations are *trees*

We can prove things about trees using induction



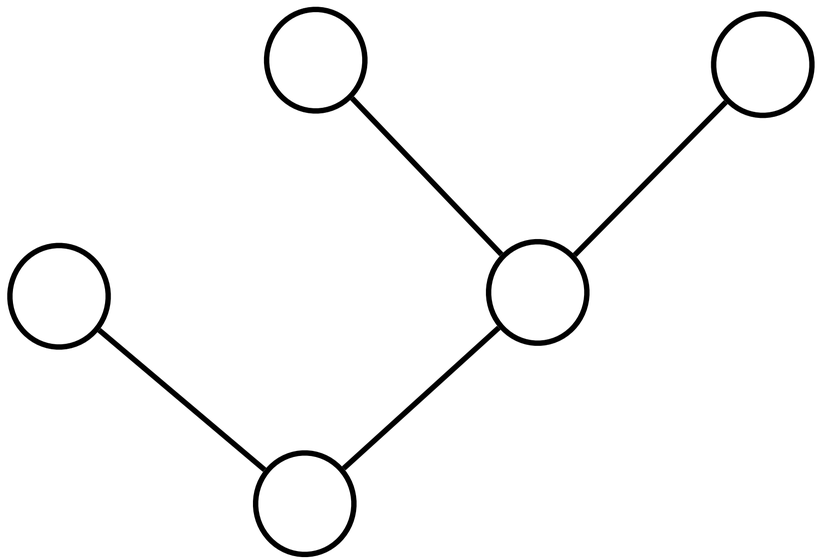
The Key Idea

$$\dfrac{\dfrac{}{\{\} \vdash \texttt{2} : \texttt{int}}\,(\text{intLit}) \qquad \dfrac{\dfrac{}{\{\texttt{y} : \texttt{int}\} \vdash \texttt{y} : \texttt{int}}\,(\text{var}) \qquad \dfrac{}{\{\texttt{y} : \texttt{int}\} \vdash \texttt{y} : \texttt{int}}\,(\text{var})}{\{\texttt{y} : \texttt{int}\} \vdash \texttt{y} + \texttt{y} : \texttt{int}}\,(\text{intAdd})}{\{\} \vdash \texttt{let y = 2 in y} + \texttt{y} : \texttt{int}}\,(\text{let})$$

Derivations are *trees*

We can prove things about trees using induction

We can prove things about *derivable judgments* using induction



The Key Idea

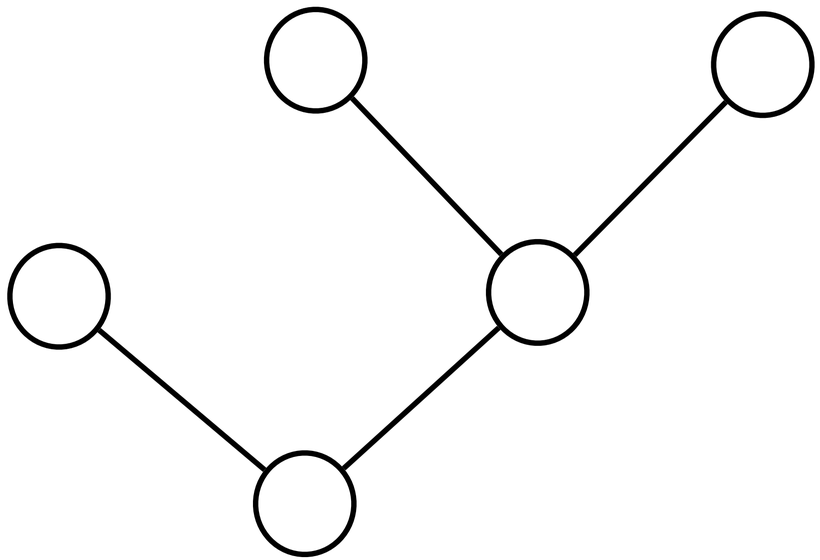
$$\dfrac{\dfrac{}{\{\} \vdash \texttt{2} : \texttt{int}}\,(\text{intLit}) \qquad \dfrac{\dfrac{}{\{\texttt{y} : \texttt{int}\} \vdash \texttt{y} : \texttt{int}}\,(\text{var}) \qquad \dfrac{}{\{\texttt{y} : \texttt{int}\} \vdash \texttt{y} : \texttt{int}}\,(\text{var})}{\{\texttt{y} : \texttt{int}\} \vdash \texttt{y} + \texttt{y} : \texttt{int}}\,(\text{intAdd})}{\{\} \vdash \texttt{let y = 2 in y} + \texttt{y} : \texttt{int}}\,(\text{let})$$

Derivations are *trees*

We can prove things about trees using induction

We can prove things about *derivable judgments* using induction

Important: Every derivable judgment corresponds to a derivation



Warm-up: Binary Trees

```
type 'a tree =
  | Empty
  | Node of 'a tree * 'a * 'a tree
```

Warm-up: Binary Trees

```
type 'a tree =
  | Empty
  | Node of 'a tree * 'a * 'a tree
```

Let $\mathsf{size}(T)$ denote the number of `Node`s and let $\mathsf{height}(T)$ denote the length of the *longest* path from the root to `Empty`

Warm-up: Binary Trees

```
type 'a tree =
  | Empty
  | Node of 'a tree * 'a * 'a tree
```

Let $\mathsf{size}(T)$ denote the number of Nodes and let $\mathsf{height}(T)$ denote the length of the *longest* path from the root to Empty

Theorem. $\mathsf{size}(T) \leq 2^{\mathsf{height}(T)} - 1$ for any tree T

Warm-up: Binary Trees

```
type 'a tree =
  | Empty
  | Node of 'a tree * 'a * 'a tree
```

Let $\text{size}(T)$ denote the number of Nodes and let $\text{height}(T)$ denote the length of the *longest* path from the root to Empty

Theorem. $\text{size}(T) \leq 2^{\text{height}(T)} - 1$ for any tree T

Proof. By induction on the *structure of trees*

Base Case: Empty

$$0 = s(\phi) \overset{?}{\leq} 2^{h(\phi)} - 1$$

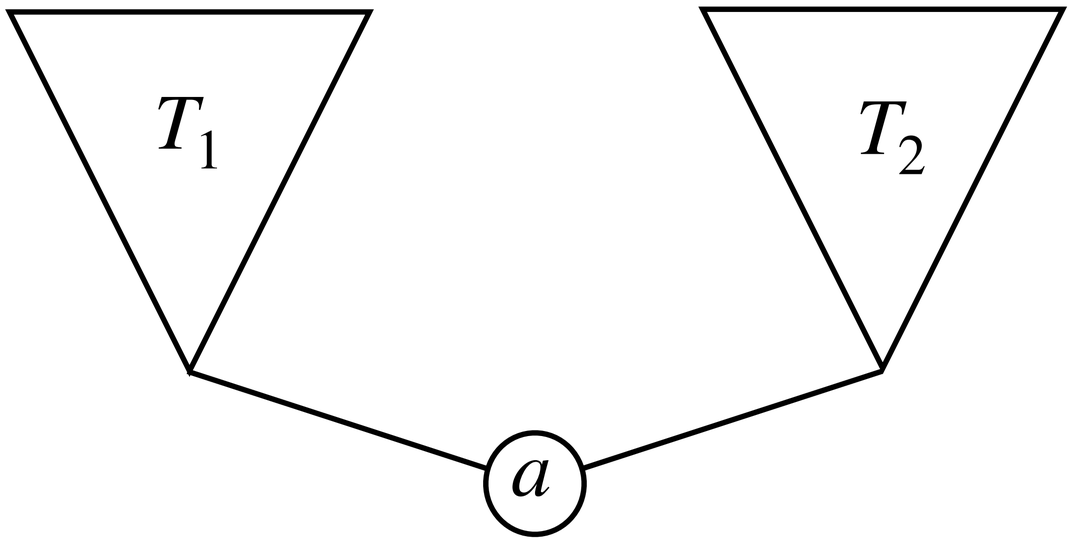
$$= 2^0 - 1$$

$$= 1 - 0$$

$$= 0$$

$$0 \leq 0 \quad \checkmark$$

Inductive Hypothesis



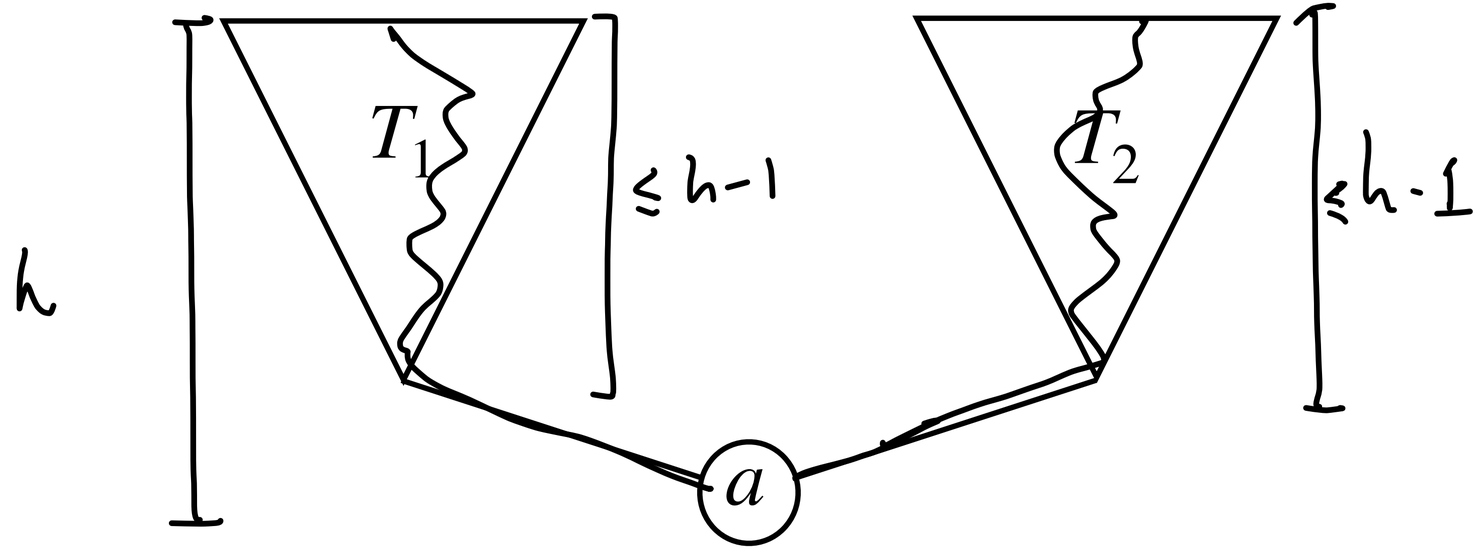
If T is of the form `Node` $(T_1,\ a,\ T_2)$ then $\text{size}(T_1) \leq 2^{\text{height}(T_1)} - 1$ and $\text{size}(T_2) \leq 2^{\text{height}(T_2)} - 1$

That is, we get to assume that what we want holds of our subtrees

Inductive Step: Nodes

WTS:

$$2^{h(T_1)} + 2^{h(T_2)} \leq 2^{h(T)}$$



$$2^{h(T_1)} + 2^{h(T_2)} \leq 2\left(2^{h(T)-1}\right) = 2^{h(T)}$$

$$\begin{aligned} s(T) &= 1 + s(T_1) + s(T_2) && \text{(def)} \\ &\leq 1 + 2^{h(T_1)} - 1 + 2^{h(T_2)} - 1 \\ &= 2^{h(T_1)} + 2^{h(T_2)} - 1 \leq 2^{h(T)} - 1 \end{aligned}$$

Another Warm-up: Well-Scopedness

Another Warm-up: Well-Scopedness

An expression e is **well-scoped** with respect to a context Γ if $x \in FV(e)$ implies x appears in Γ

Another Warm-up: Well-Scopedness

An expression e is **well-scoped** with respect to a context Γ if $x \in FV(e)$ implies x appears in Γ

$\emptyset \vdash e : \tau$

Theorem. If e is well-typed in Γ, then e is well-scoped

Another Warm-up: Well-Scopedness

An expression e is **well-scoped** with respect to a context Γ if $x \in FV(e)$ implies x appears in Γ

Theorem. If e is well-typed in Γ, then e is well-scoped

Proof. By induction on *derivations*

Base Case: Axioms

$$\frac{}{\Gamma \vdash \bullet : \top}\ \texttt{unit}$$

$FV(\bullet) = \{\}$ ✓

$$\frac{(x : \tau) \in \Gamma}{\Gamma \vdash x : \tau}\ \texttt{variable}$$

$FV(x) = \{x\}$ ✓

We need to show that expressions typed using just axioms satisfy well-scopedness

Inductive Hypothesis

$$\frac{\begin{array}{c}\vdots\\ \mathscr{D}_1\\ \Gamma_1 \vdash e_1 : \tau_1\end{array} \quad \begin{array}{c}\vdots\\ \mathscr{D}_2\\ \Gamma_2 \vdash e_2 : \tau_2\end{array} \quad \ldots \quad \begin{array}{c}\vdots\\ \mathscr{D}_k\\ \Gamma_k \vdash e_k : \tau_k\end{array}}{\Gamma \vdash e : \tau}$$

If $e_1, \ldots, e_k$ *are well-scoped (because they are typeable in the each of their contexts)*

Inductive Step 1: Application

$$\dfrac{\begin{array}{c}\vdots \\ \mathscr{D}_1 \\ \Gamma \vdash e_1 : \tau \to \tau' \end{array} \quad \begin{array}{c}\vdots \\ \mathscr{D}_2 \\ \Gamma \vdash e_2 : \tau \end{array}}{\Gamma \vdash e_1 e_2 : \tau'}\ \textbf{application}$$

By IH ①, $FV(e_1) \subseteq \Gamma$

By IH ②, $FV(e_2) \subseteq \Gamma$

$FV(e_1 e_2) = FV(e_1) \cup FV(e_2)$

$\subseteq \Gamma$

by IH ① and IH ②

What if the last rule I applied was application?

Inductive Step 2: Abstraction

$$\frac{\begin{array}{c}\vdots \\ \mathscr{D} \\ \textcircled{1}\ \Gamma, x : \tau \vdash e : \tau'\end{array}}{\Gamma \vdash \lambda x^{\tau}.\, e : \tau \to \tau'}\ \textbf{abstraction}$$

By IH①, $FV(e) \subseteq \Gamma, x : \tau$

$FV(\lambda x^{\tau}.\, e) = FV(e) \setminus \{x\} \subseteq \Gamma$ ✓

What if the last rule I applied was abstraction?

Progress and Preservation

Recall: Type Safety

Recall: Type Safety

Thm. If $\cdot \vdash e : \tau$ and there is a value v such that $\langle \varnothing, e \rangle \Downarrow v$ then $\cdot \vdash e(v) : \tau$

Recall: Type Safety

> **Thm.** If $\cdot \vdash e : \tau$ and there is a value v such that $\langle \varnothing, e \rangle \Downarrow v$ then $\cdot \vdash e(v) : \tau$

With small-step semantics, we can give a finer-grained analysis:

Recall: Type Safety

> **Thm.** If $\cdot \vdash e : \tau$ and there is a value v such that $\langle \varnothing, e \rangle \Downarrow v$ then $\cdot \vdash e(v) : \tau$

With small-step semantics, we can give a finer-grained analysis:

> **Thm.** If $\cdot \vdash e : \tau$, then
>
> » *(progress)* either e is a value or there is an e' such that $e \longrightarrow e'$
>
> » *(preservation)* If $\cdot \vdash e : \tau$ and $e \longrightarrow e'$ then $\cdot \vdash e' : \tau$

no stuck terms

Recall: Type Safety

Thm. If $\cdot \vdash e : \tau$ and there is a value v such that $\langle \varnothing, e \rangle \Downarrow v$ then $\cdot \vdash e(v) : \tau$

With small-step semantics, we can give a finer-grained analysis:

Thm. If $\cdot \vdash e : \tau$, then

doesn't imply termination

» *(progress)* either e is a value or there is an e' such that $e \longrightarrow e'$

» *(preservation)* If $\cdot \vdash e : \tau$ and $e \longrightarrow e'$ then $\cdot \vdash e' : \tau$

These results are *fundamental*. They tell us that our PL is well-behaved (it's a "good" PL)

Recall: Type Safety

> **Thm.** If $\cdot \vdash e : \tau$ and there is a value v such that $\langle \varnothing, e \rangle \Downarrow v$ then $\cdot \vdash e(v) : \tau$

With small-step semantics, we can give a finer-grained analysis:

goal for today

> **Thm.** If $\cdot \vdash e : \tau$, then
>
> » *(progress)* either e is a value or there is an e' such that $e \longrightarrow e'$
>
> » *(preservation)* If $\cdot \vdash e : \tau$ and $e \longrightarrow e'$ then $\cdot \vdash e' : \tau$

These results are *fundamental*. They tell us that our PL is well-behaved (it's a "good" PL)

Disclaimer: We're gonna hand-wave liberally

Recall: STLC

Value

$\bullet$ or $\lambda x^{\tau}.e$

Syntax

$$e ::= \bullet \mid x \mid \lambda x^{\tau}.e \mid ee$$

$$\tau ::= \top \mid \tau \to \tau$$

$$x ::= \textit{variables}$$

Typing

$$\frac{}{\Gamma \vdash \bullet : \top}\ \texttt{unit}$$

$$\frac{\Gamma, x : \tau \vdash e : \tau'}{\Gamma \vdash \lambda x^{\tau}.e : \tau \to \tau'}\ \texttt{abstraction}$$

$$\frac{(x : \tau) \in \Gamma}{\Gamma \vdash x : \tau}\ \texttt{variable}$$

$$\frac{\Gamma \vdash e_1 : \tau \to \tau' \qquad \Gamma \vdash e_2 : \tau}{\Gamma \vdash e_1 e_2 : \tau'}\ \texttt{application}$$

Semantics

$$\frac{e_1 \longrightarrow e_1'}{e_1 e_2 \longrightarrow e_1' e_2}\ \texttt{leftEval}$$

$$\frac{}{(\lambda x.e)e' \longrightarrow [e'/x]e}\ \texttt{beta}$$

Progress (STLC)

$\emptyset \vdash e : \tau$

Theorem. If e is well-typed then e is a value or there is an expression e' such that $e \longrightarrow e'$

Proof. By induction over *derivations*

Base Case: Axioms

$$\frac{}{\cdot \vdash \bullet : \mathsf{T}}\ \texttt{unit}$$

already value

impossible

$$\frac{(x : \tau) \in \varnothing}{\cdot \vdash x : \tau}\ \texttt{variable}$$

impossible by well-scoped trees

We need to show that expressions typed using just axioms yield non-stuck terms

Inductive Step 1: Application

$$\frac{\overset{1}{\cdot \vdash e_1 : \tau \to \tau'} \qquad \cdot \vdash e_2 : \tau}{\cdot \vdash e_1 e_2 : \tau'} \text{ application}$$

⋮

1 e_1 is not a value. by IH 1 there's a e_1' s.t. $e_1 \to e_1'$

so

$$\frac{e_1 \to e_1'}{e_1 e_2 \to e_1' e_2} \text{ (1E)}$$

2 e_1 is a value

i $e_1 = \bullet$, not possible

ii $e_1 = \lambda x^{\tau}.e$ then

(by type uniqueness)

$$\frac{\cdot \vdash e_1 : \tau \to \tau_2 \quad \text{and} \quad \cdot \vdash \bullet : T}{(\lambda x^{\tau}.e) e_2 \to [e_2/x]e} \text{ (beta)}$$

*What do we know given that e_1 is either a **value** or **reducible**?*

Inductive Step 2: Abstraction

$$\frac{\{x : \tau\} \vdash e : \tau'}{\cdot \vdash \underbrace{\lambda x^{\tau} . e}_{\text{already a value}} : \tau \to \tau'} \quad \texttt{abstraction}$$

Our expression already a value if the last rule we applied was abstraction!

Preservation (STLC)

Theorem. If $\Gamma \vdash e : \tau$ and $e \longrightarrow e'$ then $\Gamma \vdash e' : \tau$

Proof. By induction over *derivations*

This one is much tricker...

Base Case: Axioms

$$\frac{}{\Gamma \vdash \underbrace{\bullet}_{\text{value}} : \top}\ \texttt{unit}$$

$$\frac{(x : \tau) \in \Gamma}{\Gamma \vdash \underbrace{x : \tau}_{\text{no step possible}}}\ \texttt{variable}$$

Expressions typed using just axioms cannot be reduced (nothing to do here)

Inductive Step 1: Abstraction

$$\frac{\Gamma, x : \tau \vdash e : \tau'}{\Gamma \vdash \underbrace{\lambda x^{\tau} . e}_{\text{already value}} : \tau \to \tau'} \quad \texttt{abstraction}$$

Expressions derived using abstraction as the last rule is already a value (nothing to do here)

Inductive Step 2: Application

$$\frac{\Gamma \vdash e_1 : \tau \to \tau' \qquad \Gamma \vdash e_2 : \tau}{\Gamma \vdash e_1 e_2 : \tau'} \text{ application}$$

This is where the work comes in...

The trick: We do induction (inside our current induction) on the structure of *semantic* derivations!

What possible ways can $e_1 e_2$ *be reduced?*

Inductive Step 2.1: leftEval

$$\frac{e_1 \longrightarrow e_1'}{e_1 e_2 \longrightarrow e_1' e_2}\ \texttt{leftEval}$$

$$\frac{\overset{1}{\Gamma \vdash e_1 : \tau \to \tau'} \qquad \Gamma \vdash e_2 : \tau}{\Gamma \vdash e_1 e_2 : \boxed{\tau'}}\ \texttt{application}$$

$\Gamma \vdash e_1 : \tau \to \tau'$ and $e_1 \to e_1'$ by IH $\overset{1}{\Gamma \vdash e_1' : \tau \to \tau'}$

$$\frac{\Gamma \vdash e_1' : \tau \to \tau' \qquad \Gamma \vdash e_2 : \tau}{\Gamma \vdash e_1' e_2 : \boxed{\tau'}}$$

What if our last rule was an application ***and*** $e_1 e_2$ *is reducible by leftEval?*

Inductive Step 2.1: leftEval

$$\frac{\Gamma, x:\tau \vdash e:\tau'}{\Gamma \vdash (\lambda x^{\tau}.\, e) : \tau \to \tau'} \qquad \Gamma \vdash e_2 : \tau \; ①$$

$$\frac{}{(\lambda x^{\tau}.\, e)e_2 \longrightarrow [e_2/x]e}\ \textbf{beta} \qquad \frac{\Gamma \vdash (\lambda x^{\tau}.\, e) : \tau \to \tau' \qquad \Gamma \vdash e_2 : \tau}{\Gamma \vdash (\lambda x^{\tau}.\, e)e_2 : \boxed{\tau'}}\ \textbf{application}$$

$\Gamma, x:\tau \vdash e:\tau'$ and $\Gamma \vdash e_2 : \tau$ by ① then

by subst. lem $\Gamma \vdash [e_2/x]e : \boxed{\tau'}$

What if our last rule was an application ***and*** $e_1 e_2$ *is reducible by beta?*

Substitution Lemma

$\{y : int, x : int\} \vdash \boxed{x} + 1 : int$

$\{y : int\} \vdash \boxed{y + 2} : int$

$\{y : int\} \vdash (y+2) + 1 : int$

Lemma. If $\Gamma \vdash e_2 : \tau_2$ and $\Gamma, x : \tau_2 \vdash e : \tau$ then

$$\Gamma \vdash [e_2/x]e : \tau$$

That is, if e is well-typed in a context with (x, τ) then we can substitute x with anything of type τ and it's still the same type

(we can prove this by, you guessed it, induction on derivations)

The Point

```
let rec eval env e =
  match e with
  | Var x -> Env.find x env
  ...
```

Progress and preservation tell us that **terms never get stuck during evaluation**

This is huge. I can't emphasize this enough

Our type system ensures we only evaluate programs that make sense!

Summary

Progress and **preservation** are fundamental features of good programming languages

We can prove things about well-typed expressions by performing **induction** over derivations